DEVELOPER SESSION • 2026

Claude Code

Der neue Standard

Wie KI-Agenten die Regeln der Softwareentwicklung neu definieren

STATUS QUO

KI schreibt bereits *die Hälfte des Codes*

des GitHub-Codes ist KI-generiert oder KI-unterstützt

Commits pro Woche auf GitHub

mehr Pull Requests pro Entwickler mit KI-Tools

1. APRIL 2026 • CLAUDE CODE SOURCE-LEAK

512.000 Zeilen Code.
Aus Versehen veröffentlicht.

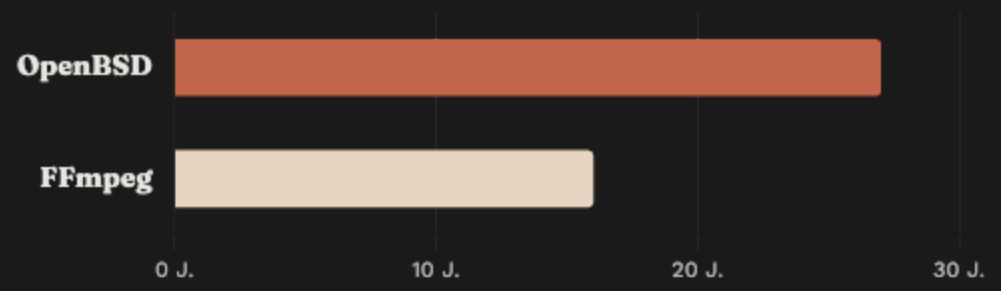
1. Ein fehlender Eintrag in **.npmignore** legte den gesamten Claude Code TypeScript-Quellcode offen
2. Über 41.500 Forks in Stunden. **Schnellstes Repo auf 50K Stars** in der Geschichte von GitHub
3. Entdeckt: **KAIROS**, ein permanent aktiver Hintergrund-Agent, der autonom im Hintergrund arbeitet
4. Noch nicht geshippt: konversationsübergreifendes Gedächtnis, Multi-Agent-Kollaboration, Session Review

7. APRIL 2026 • PROJECT GLASSWING • 6 TAGE SPÄTER

Claude Mythos

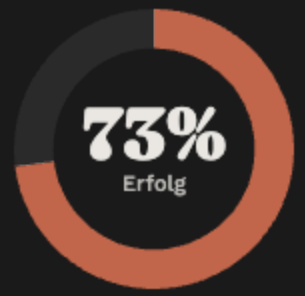
Tausende Zero-Day-Schwachstellen in **allen großen Betriebssystemen und Browsern** gefunden

BEISPIELE



Nur zwei Beispiele von Tausenden entdeckten Schwachstellen.

UK AI SECURITY INSTITUTE



73% Erfolg bei Experten-CTFs (Capture the Flag: simulierte Cyberangriffe). Vor April 2025 schaffte das kein einziges KI-Modell.

Nicht öffentlich verfügbar. Nur für AWS, Apple, Google, Microsoft und ausgewählte Partner.

DIE ENTWICKLUNG

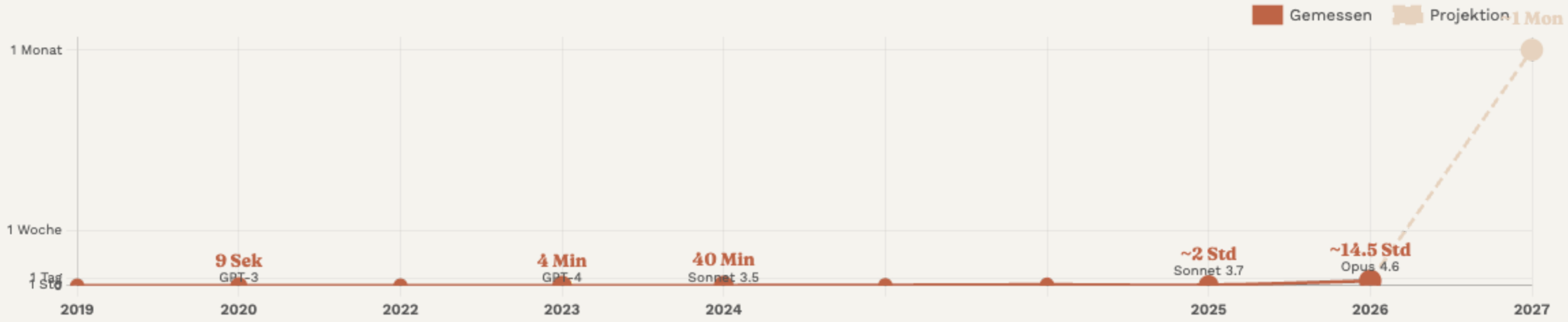
Die drei Stufen

Coding → **Tool Use** → **Computer Use**

Die Modelle werden zwar besser, aber der größte Fortschritt kommt durch **bessere Werkzeuge**. Nicht das Modell allein macht den Unterschied, sondern was es benutzen kann.

METR • TASK-ZEITHORIZONTE

Was KI-Agenten
heute schon können

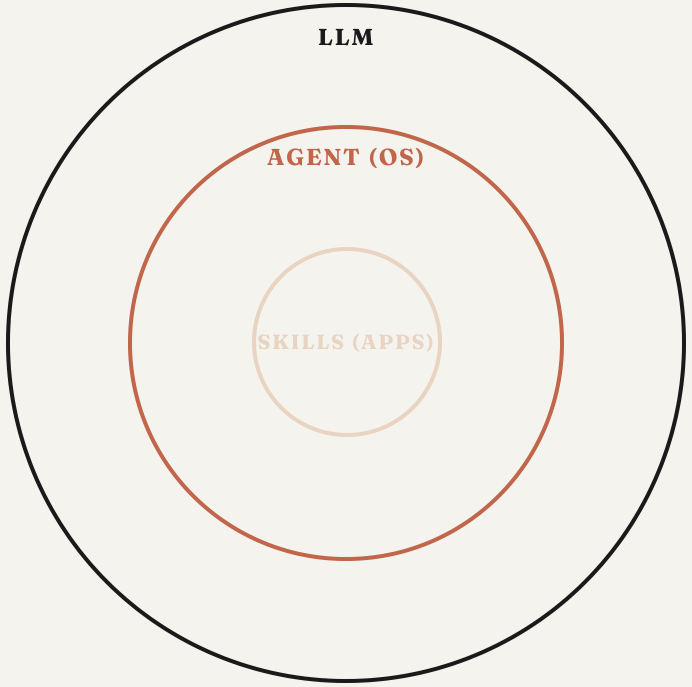


Quelle: METR — metr.org/time-horizons • Verdopplung alle ~7 Monate, zuletzt alle 4 Monate

TEIL ZWEI

Wie Claude Code

wirklich funktioniert



AGENT-ARCHITEKTUR

Das Harness-Modell

DER LOOP

Tools **(read, list, write)** liefern Kontext → das LLM erkennt Probleme und nimmt Änderungen vor → bekommt Feedback → iteriert weiter.

Lesen → **Denken** → **Handeln** → **Feedback** → **Wiederholen**

Das ist kein Autocomplete. Claude sagt nicht eure nächste Zeile voraus. Es **denkt über euer Problem nach**, probiert Lösungen aus und korrigiert sich selbst.

ampcode.com/how-to-build-an-agent • mihaileric.com/The-Emperor-Has-No-Clothes

KERNERKENNTNIS

Der Loop ist nur so gut
wie der Kontext, den du ihm gibst

WENIG KONTEXT

Einzeiler-Prompt, keine Projektinfos, keine Konventionen

→ **Falsche Annahmen, falscher Stil, over-engineered**

VIEL KONTEXT

CLAUDE.md, klare Ziele, euer Stack und eure Konventionen

→ **Code, der perfekt zu eurem Projekt passt**

TEIL DREI

CLAUDE.md

Das Gehirn deines Projekts

DAS FUNDAMENT

Was ist CLAUDE.md?

1. Eine Markdown-Datei **im Root eures Repos**, die Claude sagt, wie es sich in eurem Projekt verhalten soll
2. **Versioniert** im Repo. Euer ganzes Team teilt und reviewt sie in PRs
3. **Bleibt über Sessions bestehen.** Einmal schreiben, Claude hält sich immer dran
4. Kein System-Prompt. Sie ist **projektspezifisch**, kombinierbar und lebt in eurer Codebase
5. Startet mit Karpathys Vorlage: `claude install-skill https://github.com/forrestchang/andrej-karpathy-skills`

GITHUB.COM/FORRESTCHANG/ANDREJ-KARPATHY-SKILLS

So schreibt ihr eine State-of-the-Art CLAUDE.md

01

Erst denken, dann coden

„Mehrdeutigkeiten explizit benennen. Mehrere Interpretationen präsentieren, statt still eine zu wählen.“

02

Einfachheit zuerst

Unnötige Abstraktionen vermeiden. Der Test: Würde ein Senior Engineer es als überkompliziert bezeichnen?

GITHUB.COM/FORRESTCHANG/ANDREJ-KARPATHY-SKILLS

So schreibt ihr eine State-of-the-Art CLAUDE.md

03

Chirurgische Änderungen

Bestehenden Stil beibehalten. Jede geänderte Zeile muss direkt auf die Anforderung zurückführbar sein. Keine Refactors im Vorbeigehen.

04

Zielorientierte Ausführung

Vage Anforderungen in testbare Kriterien übersetzen. Nicht „mach es besser“ sagen, sondern klar definieren, wie „fertig“ aussieht.

TEIL VIER

Skills

Modulare Workflows

CODE ALS SCHNITTSTELLE

Was sind Skills?

Statt aufgabenspezifische Agenten zu bauen, behandelt Claude Code Code als **„Schnittstelle zur digitalen Welt“**. Ein universeller Agent, angereichert mit modularen Skills.

1. Jeder Skill ist eine **SKILL.md**-Datei in einem Ordner, reines Markdown
2. Installieren: **claude skills add mattpocock/skills/grill-me**
3. Ausführen: **/grill-me** in der Konversation eingeben
4. Eigene Skills schreiben dauert **keine 10 Minuten**

ZUM AUSPROBIEREN

Ressourcen

KARPATHYS CLAUDE.MD INSTALLIEREN

```
claude install-skill https://github.com/forrestchang/andrej-karpathy-skills
```

GRILL ME SKILL (IDEEN STRESSTESTEN)

```
npx skills@latest add mattpocock/skills/grill-me
```

IMPECCABLE (FRONTEND DESIGN SKILLS)

```
npx skills add pbakaus/impeccable
```

EVERYTHING CLAUDE CODE (SKILLS, HOOKS, AGENTS, SECURITY AUS EINER QUELLE)

```
github.com/affaan-m/everything-claude-code
```

POWER-PROMPTS

`Find the root cause of the issue` `Implement the plan with painstaking detail` `Make changes doing minimal essential changes`

SHORTCUTS & BEFEHLE

Übersicht Claude Code

Neuer Chat	`Cmd+K`	Letzte Nachricht bearbeiten	`↑`
Generierung stoppen	`Cmd+.`	Sidebar ein/ausblenden	`Cmd+/`
Neue Zeile ohne Senden	`Shift+Enter`	Zu Checkpoint zurückspulen	`Esc Esc`
Permission-Modus wechseln	`Shift+Tab`	Nebenfrage stellen	`/btw`

EUER SPICKZETTEL

Drei Dinge, die ihr *heute noch tun solltet*

1 CLAUDE.md anlegen, falls noch nicht vorhanden

Nutzt die Karpathy CLAUDE.md als Vorlage. Stack, Konventionen, Testbefehle rein. Sofort bessere Ergebnisse.

2 Skills installieren und ausprobieren

Impeccable für Frontend, Grill Me zum Ideen stresstesten. Spart euch wiederkehrende Prompts.

3 Mehrere Claude-Instanzen parallel nutzen

Kontext nicht überladen. Immer nur ein Thema pro Instanz. Lieber drei fokussierte Sessions als eine chaotische.